

Chapter 50

USB 2.0 Full-Speed OTG

50.1 Overview

The ESP32-P4 features a USB 2.0 Full-Speed On-The-Go peripheral (henceforth referred to as OTG_FS) along with integrated transceivers. This OTG_FS conforms to USB 2.0 specification, OTG Revision 1.3, and OTG Revision 2.0 specifications, OTG_FS can operate as either a USB Host or Device and supports 12 Mbit/s full-speed (FS) and 1.5 Mbit/s low-speed (LS) data rates of the USB 2.0 specification. The Host Negotiation Protocol (HNP) and the Session Request Protocol (SRP) are also supported.

50.2 Glossary

The following abbreviations and terms are used in this chapter.

Host	The host computer system where the USB Host Controller is installed. This includes the host hardware platform (CPU, bus, etc.) and the operating system in use.
Device	A logical or physical entity that performs a function. The actual entity described depends on the context of the reference. At the lowest level, device may refer to a single hardware component, as in a memory device. At a higher level, it may refer to a collection of hardware components that perform a particular function, such as a USB interface device. At an even higher level, device may refer to the function performed by an entity attached to the USB; for example, a data/FAX modem device. When used as a non-specific reference, a USB device is either a hub or a function.
Full-speed	USB operation at 12 Mbit/s.
Low-speed	USB operation at 1.5 Mbit/s.
Host Negotiation Protocol (HNP)	Allows the host function to be transferred between two directly connected OTG devices and eliminates the need for a user to switch the cable connections in order to allow a change in control of communications between the devices.
Session Request Protocol (SRP)	Allows a B-device to request the A-device to turn on the power supply to the USB interface (VBUS) and start a session.
Endpoint	A uniquely addressable portion of a USB device that is the source or sink of information in a communication flow between the host and device.
Scatter/Gather DMA mode	Accesses discontinuous memory areas via DMA.

Buffer DMA mode	Accesses contiguous memory areas via DMA.
Slave mode	Accesses memory area through the CPU.
RX FIFO	Stores the received data.
TX FIFO	Stores the data to be transmitted.
Periodic FIFO	Stores Isochronous transfer data and Interrupt transfer data to be sent.
Non-periodic FIFO	Stores Control transfer data and Bulk transfer data to be sent.

50.3 Features

50.3.1 General Features

- USB 2.0 specification, OTG Revision 1.3 and OTG Revision 2.0 specifications
- USB 2.0 full-speed and low-speed data rates
- HNP and SRP as A-device or B-device
- Dynamic FIFO (DFIFO) sizing, maximum to 1 KB
- Multiple modes of memory access
 - Scatter/Gather DMA mode
 - Buffer DMA mode
 - Slave mode
- Two integrated transceivers

50.3.2 Device Mode Features

- Endpoint 0 always present, bi-directional, consisting of EPO IN and EPO OUT
- Six additional endpoints 1~6, configurable as IN or OUT
- Maximum of five IN endpoints concurrently active at any time, including EPO IN
- All OUT endpoints share a single RX FIFO
- Each IN endpoint has a dedicated TX FIFO

50.3.3 Host Mode Features

- Eight host channels
- RX FIFO: shared by all periodic and non-periodic transactions
- Two TX FIFO:
 - One shared by all non-periodic transactions
 - One shared by all periodic transactions
- All of the above FIFOs share a 1 KB RAM.
- The size of each FIFO is configurable, with a maximum of 1 KB.

50.4 Functional Description

50.4.1 Controller Core and Interfaces

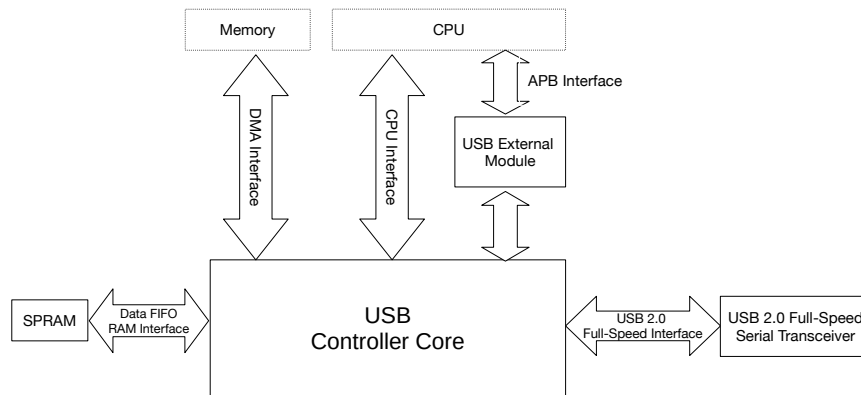


Figure 50.4-1. OTG_FS System Architecture

The core part of the OTG_FS peripheral is the USB Controller Core. The controller core has the following interfaces, see Figure 50.4-1:

- **CPU Interface**

Provides the CPU with read/write access to the controller core's various registers and FIFOs. This interface is internally implemented as an AHB slave interface. The way to access the FIFOs through the CPU interface is called Slave mode.

- **APB Interface**

Allows the CPU to control the USB controller core via the USB external controller.

- **DMA Interface**

Provides the controller core's internal DMA with read/write access to system memory, e.g., fetching and writing data payloads in DMA mode. This interface is internally implemented as an AHB master interface.

- **USB 2.0 Full-Speed Interface**

ESP32-P4 has two full-speed transceivers. This interface is used to connect the controller core to one of USB 2.0 full-speed serial transceivers. Aside from USB OTG, ESP32-P4 also includes a USB Serial/JTAG controller, see Chapter 51 [USB Serial/JTAG Controller \(USB_SERIAL_JTAG\)](#). These two USB controllers can utilize the integrated internal transceivers by time-division multiplexing. In the following sections, GPIO24 and GPIO25 are referred to as FS_PHY1, while GPIO26 and GPIO27 as FS_PHY2.

By default FS_PHY1 connects to USB Serial/JTAG controller and FS_PHY2 to OTG_FS. The connection is configurable via eFuse settings:

- 0: FS_PHY1 is connected to USB Serial/JTAG controller, while FS_PHY2 to OTG_FS.
- 1: FS_PHY2 is connected to USB Serial/JTAG controller, while FS_PHY1 to OTG_FS.

- **USB External Controller**

The USB External Controller is primarily used to control the routing of the USB 2.0 full-speed serial interface to the internal registers, thus allowing the CPU to perform query operations, etc. The External Controller can also enable a power saving mode by gating the controller core's clock, i.e., AHB clock, or

powering down the connected SPRAM clock. Note that this power saving mode is different from the power savings via SRP.

- **Data FIFO RAM Interface**

The multiple FIFOs used by the controller core are not actually located within the controller core itself, but on the Single-Port RAM (SPRAM). FIFOs are dynamically sized, thus are allocated at run-time in the SPRAM. When the CPU, DMA, or the controller core attempts to read/write to FIFOs, those accesses are routed through the data FIFO RAM interface.

50.4.2 Memory Layout

Figure 50.4-2 illustrates the memory layout of the registers which are used to configure and control the USB Controller Core. Note that USB External Controller uses a separate set of registers called wrap registers.

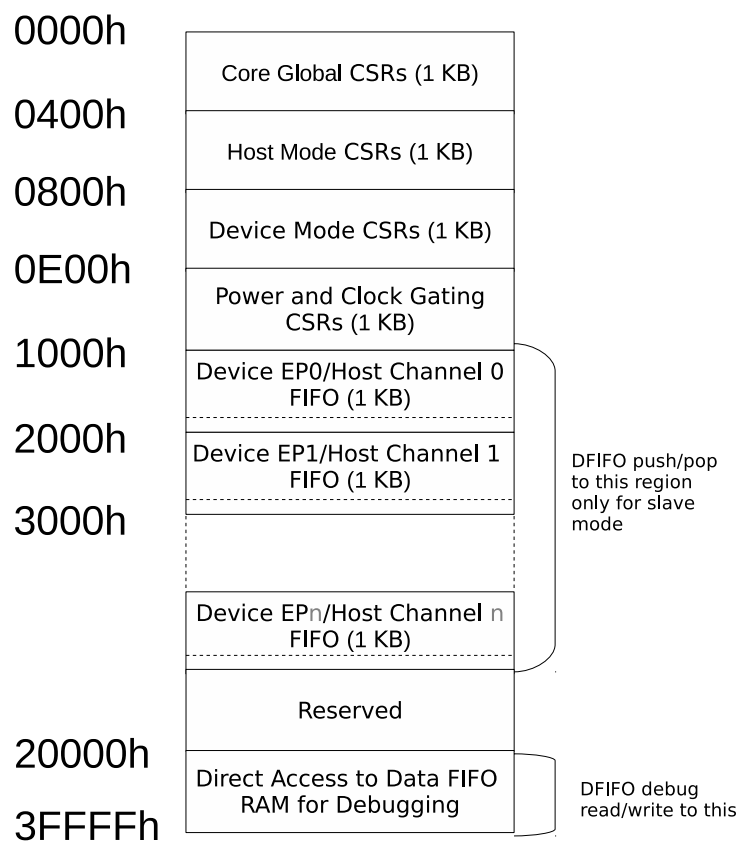


Figure 50.4-2. OTG_FS Register Layout

50.4.2.1 Control & Status Registers (CSRs)

- **Core Global CSRs**

These registers are responsible for the configuration/control/status of the global features of OTG_FS, i.e., features which are common to both Host and Device modes. These features include:

- OTG control (HNP, SRP, and A/B-device detection);
- USB configuration (Host or Device mode selection and PHY selection);
- and system-level interrupts.

Software can access these registers whilst in Host or Device modes.

- **Host Mode CSRs**

These registers are responsible for the configuration/control/status in Host mode, thus should only be accessed in Host mode. Each channel has its own set of registers within the Host mode CSRs.

- **Device Mode CSRs**

These registers are responsible for the configuration/control/status in Device mode, thus should only be accessed in Device mode. Each endpoint has its own set of registers within the Device mode CSRs.

- **Power and Clock Gating Register**

A single register controls power-down and gates various clocks.

50.4.2.2 FIFO Access

The OTG_FS makes use of multiple FIFOs to buffer the data payloads to be transmitted or to buffer the received data payloads. The number and type of FIFOs are dependent on Host or Device mode, and the number of channels or endpoints used, see Section 50.4.3. There are two ways to access the FIFOs: DMA mode and Slave mode. In Slave mode, the CPU accesses these FIFOs by reading and writing to either the DFIFO push/pop regions or the DFIFO read/write debug region. FIFO access is governed by the following rules:

- Read access to any address in any one of the 1 KB push/pop regions will result in a pop from the shared RX FIFO.
- Write access to a particular 1 KB push/pop region will result in a push to the corresponding endpoint or channel's TX FIFO given that the endpoint is an IN endpoint, or the channel is an OUT channel.
 - In Device mode, data is pushed to the corresponding IN endpoint's dedicated TX FIFO.
 - In Host mode, data is pushed to the non-periodic TX FIFO or the periodic TX FIFO depending on whether the channel is a non-periodic channel, or a periodic channel.
- Access to the 128 KB read/write region will result in direct read/write instead of a push/pop. This is generally used for debugging purposes only.

Note that pushing and popping data to and from the FIFOs by the CPU is only required in Slave mode. In DMA mode, the internal DMA will handle all pushing/popping of data to and from the TX and RX FIFOs.

50.4.3 FIFO and Queue Organization

The FIFOs in OTG_FS are primarily used to hold data packet payloads, i.e., the data field of USB Data packets. TX FIFOs are used to store data payloads that will be transmitted by OUT transactions in Host mode or IN transactions in Device mode. RX FIFOs are used to store received data payloads of IN transactions in Host mode or OUT transactions in Device mode. In addition to storing data payloads, RX FIFOs also store a **status entry** for each data payload. Each status entry contains information about a data payload such as channel number, byte count, and validity status. In Slave mode, status entries are also used to indicate various channel events.

The portion of SPRAM that can be used for FIFO allocation has a depth of 256 and a width of 35 bits (32 data bits plus 3 control bits). The multiple FIFOs used by each channel in Host mode or endpoint in Device mode are allocated into the SPRAM and can be dynamically sized.

50.4.3.1 Host Mode FIFOs and Queues

The following FIFOs are used in Host mode, see Figure 50.4-3:

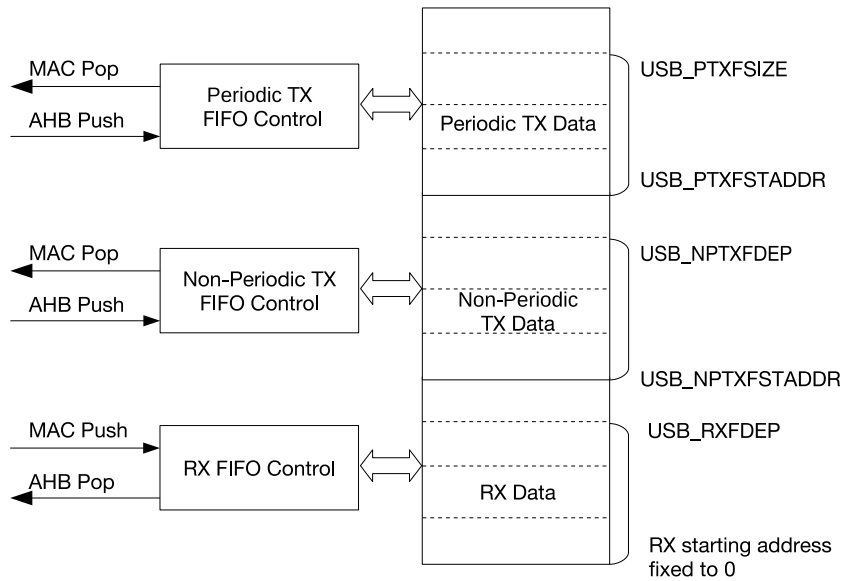


Figure 50.4-3. Host Mode FIFOs

- **Non-periodic TX FIFO:** Stores data payloads of Bulk and Control OUT transactions for all channels.
- **Periodic TX FIFO:** Stores data payloads of Interrupt or Isochronous OUT transactions for all channels.
- **RX FIFO:** Stores data payloads of all IN transactions, and status entries that are used to indicate size of data payloads and transaction/channel events such as transfer complete or channel halted.

In addition to FIFOs, Host mode also contains two request queues used to queue up the various transaction request from the multiple channels. Each entry in a request queue holds the IN/OUT channel number along with other information to perform the transaction, such as transaction type. Request queues are also used to queue other types of requests such as a channel halt request.

Unlike FIFOs, request queues are fixed in size and cannot be accessed directly by software. Rather, once a channel is enabled, requests will be automatically written to the request queue by the Host core. The order in which the requests are written into the queue determines the sequence of transactions on the USB.

Host mode contains the following request queues:

- **Non-periodic request queue:** Request queue for all non-periodic Bulk and Control channels. The queue has a depth of four entries.
- **Periodic request queue:** Request queue for all periodic Interrupt and Isochronous channels. The queue has a depth of eight entries.

When scheduling transactions, hardware will execute all requests on the periodic request queue first before executing requests on the non-periodic request queue.

50.4.3.2 Device Mode FIFOs

The following FIFOs are used in Device mode, see Figure 50.4-4:

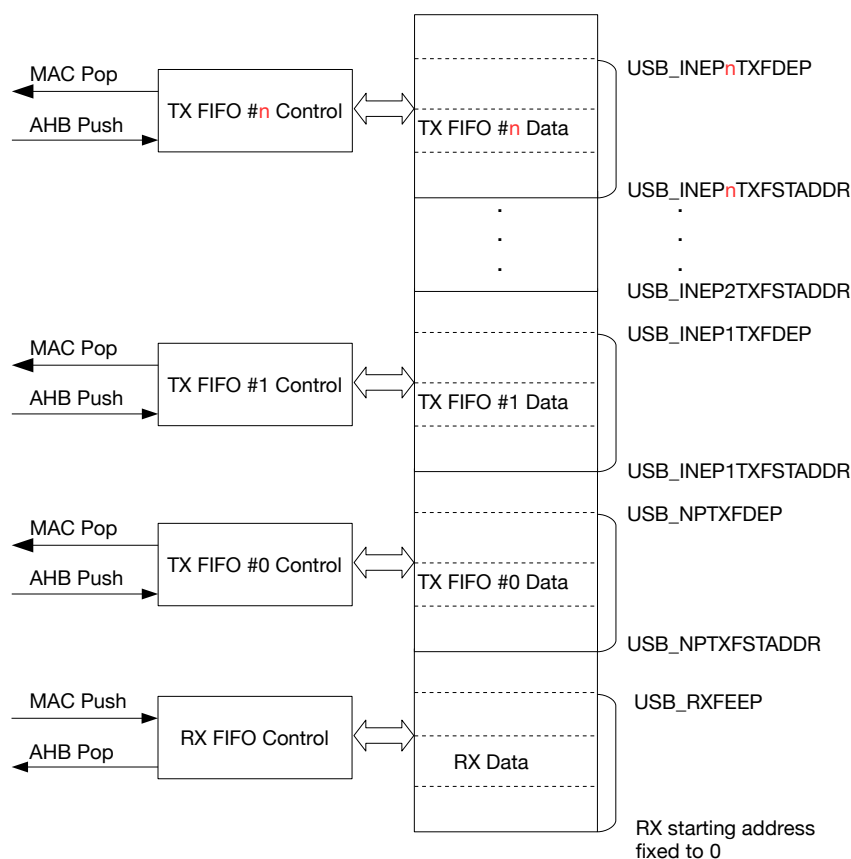


Figure 50.4-4. Device Mode FIFOs

- **RX FIFO:** Stores data payloads received in Data packet, and status entries used to indicate size of those data payloads.
- **Dedicated TX FIFO:** Each active IN endpoint will have a dedicated TX FIFO used to store all IN data payloads of that endpoint, regardless of the transaction type, both periodic and non-periodic IN transactions.

Due to the dedicated FIFOs, Device mode does not use any request queues. Instead, the order of IN transactions are determined by the Host.

50.4.4 Interrupt Hierarchy

OTG_FS provides a single interrupt line which can be routed via the interrupt matrix to one of the CPUs. The interrupt signal can be unmasked by setting USB_GLBLINTRMSK. The OTG_FS interrupt is an OR of all bits in the USB_GINTSTS_REG register, and the bits in USB_GINTSTS_REG can be unmasked by setting the corresponding bits in the USB_GINTMSK_REG register. USB_GINTSTS_REG contains system level interrupts, and also specific bits for Host or Device mode interrupts, and OTG specific interrupts. OTG_FS interrupt sources are organized as Figure 50.4-5 shows.

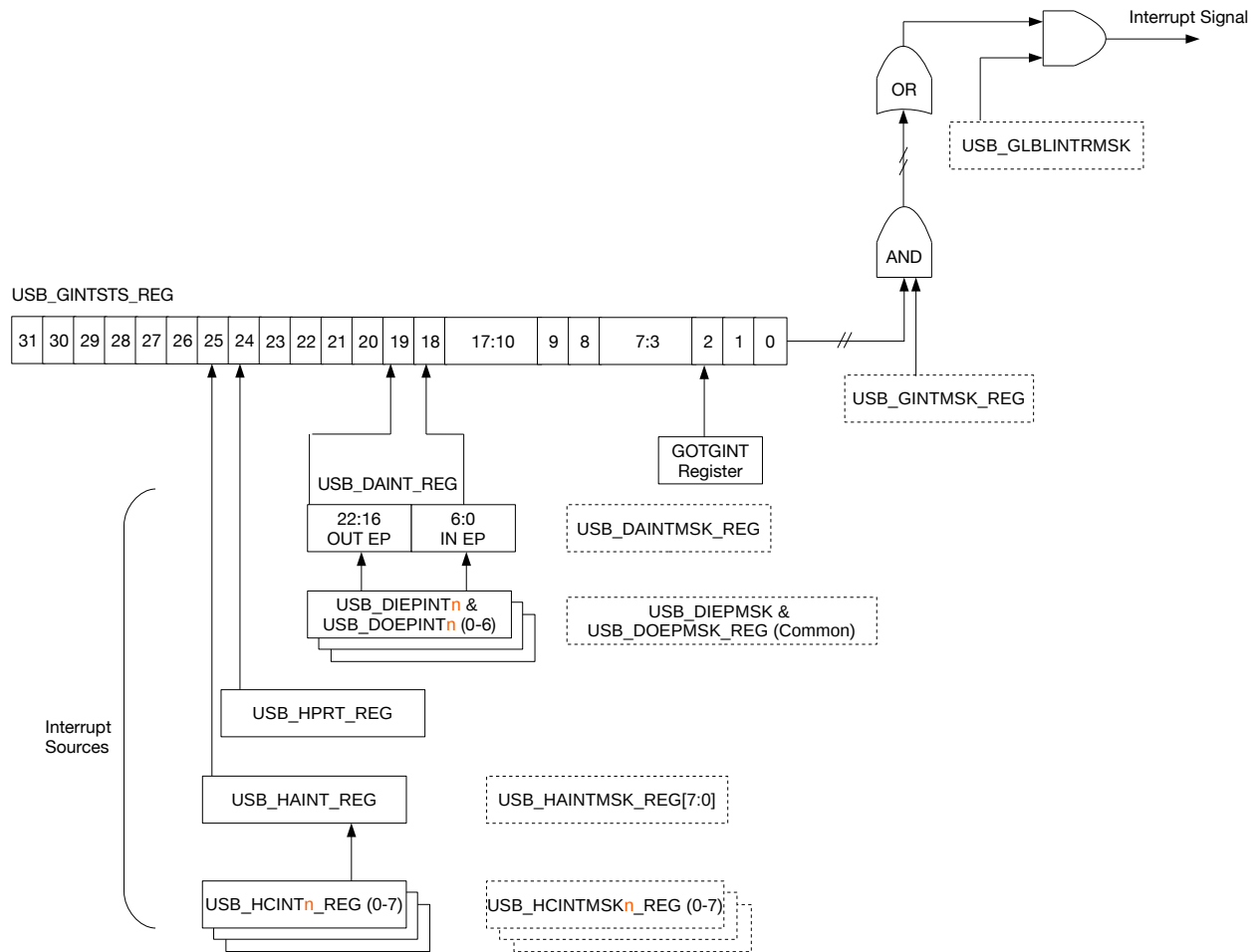


Figure 50.4-5. OTG_FS Interrupt Hierarchy

The following bits of the USB_GINTSTS_REG register indicate an interrupt source lower in the hierarchy:

- **USB_PRTINT** indicates that the Host port has a pending interrupt. The USB_HPRT_REG register indicates the interrupt source.
- **USB_HCHINT** indicates that one or more Host channels have a pending interrupt. Read the USB_HAINT_REG register to determine which channel(s) have a pending interrupt, then read the pending channel's USB_HCINT_n_REG register to determine the interrupt source.
- **USB_OEPINT** indicates that one or more OUT endpoints have a pending interrupt. Read the USB_DAINTRMsk_REG register to determine which OUT endpoint(s) have a pending interrupt, then read the USB_DOEPINT_n_REG register to determine the interrupt source.
- **USB_IEPINT** indicates that one or more IN endpoints have a pending interrupt. Read the USB_DAINTRMsk_REG register to determine which IN endpoint(s) are pending, then read the pending IN endpoint's USB_DIEPINT_n_REG register to determine the interrupt source.
- **USB_OTGINT** indicates an On-The-Go event has triggered an interrupt. Read the USB_GOTGINT_REG register to determine which OTG event(s) triggered the interrupt.

50.4.5 Slave Mode and DMA Modes

OTG_FS supports three ways to access memory: Slave mode, Buffer DMA mode, and Scatter/Gather DMA mode.

50.4.5.1 Slave Mode

In Slave mode, all data payloads must be pushed/popped to and from the FIFOs by the CPU.

- When transmitting a packet using IN endpoints or OUT channels, the data payload must be pushed into the corresponding endpoint or channel's TX FIFO.
- When receiving a packet, the packet's status entry must first be popped off the RX FIFO by reading USB_GRXSTSP_REG. The status entry should be used to determine the length of the packet's payload (in bytes). The corresponding number of bytes must then be manually popped off the RX FIFO by CPU reading from the RX FIFO's memory region.

50.4.5.2 Buffer DMA Mode

Buffer DMA mode is similar to Slave mode but utilizes the internal DMA to push and pop data payloads to the FIFOs.

- When transmitting a packet using IN endpoints or OUT channels, the data payload's address in memory should be written to USB_HCDMA n _REG in Host mode or USB_DIEPDMA n _REG in Device mode. When the endpoint or channel is enabled, the internal DMA will push the data payload from memory into the TX FIFO of the channel or endpoint.
- When receiving a packet using OUT endpoints or IN channels, the address of an empty buffer in memory should be written to USB_HCDMA n _REG in Host mode or USB_DOEPDMA n _REG in Device mode. When the endpoint or channel is enabled, the internal DMA will pop the data payload from RX FIFO into the buffer.

50.4.5.3 Scatter/Gather DMA Mode

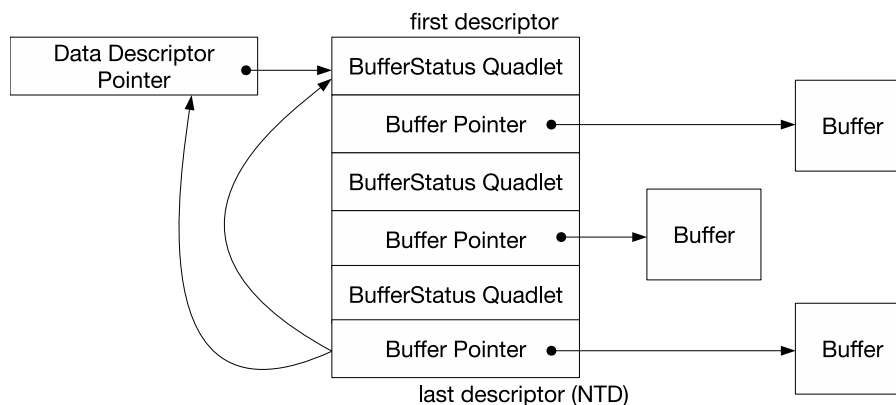


Figure 50.4-6. Scatter/Gather DMA Descriptor List

In Scatter/Gather DMA mode, buffers containing data payloads can be scattered throughout memory. Each endpoint or channel will have a contiguous DMA descriptor list, where each descriptor contains a 32-bit

pointer to the data payload or buffer and a 32-bit buffer descriptor (BufferStatus Quadlet). The data payloads and buffers can correspond to a single transaction (i.e., < 1 MPS bytes) or an entire transfer (> 1 MPS bytes). MPS: maximum packet size. The list is implemented as a ring buffer meaning that the DMA will return to the first entry when it encounters the last entry on the list.

- When transmitting a transfer/transaction using IN endpoints or OUT channels, the DMA will gather the data payloads from the multiple buffers and push them into a TX FIFO.
- When receiving a transfer/transaction using OUT endpoints or IN channels, the DMA will pop the received data payloads from the RX FIFO and scatter them to the multiple buffers pointed to by the DMA list entries.

50.4.6 Transaction and Transfer Level Operation

In either Host or Device mode, communication can operate either at the transaction level or the transfer level.

50.4.6.1 Transaction and Transfer Level Operation in DMA Mode

When operating at the transaction level in DMA mode, the transfer size is set to the size of one data packet, either a maximum packet size or a short packet size.

When operating at the transfer level in DMA Host mode, software is interrupted only when a channel has been halted. Channels are halted when their programmed transfer size has completed successfully, has received a STALL, or if there are excessive transaction errors, e.g., 3 consecutive transaction errors. In DMA Device mode, all errors are handled by the controller core itself.

50.4.6.2 Transaction and Transfer Level Operation in Slave Mode

When operating at the transaction level in Slave Mode, transfers are handled one transaction at a time. Each data payload should correspond to a single data packet, and software must determine whether a retry of the transaction is necessary based on the handshake response received on the USB, e.g., ACK or NAK.

The following table describes transaction level operation in Slave mode for both IN and OUT transactions.

Table 50.4-1. IN and OUT Transactions in Slave Mode

Host Mode	Device Mode
OUT Transactions	

Host Mode	Device Mode
1. Software specifies the size of the data packet and the number of data packets (1 data packet) in the USB_HCTSIZ_n_REG register, enables the channel, then copies the packet's data payload into the TX FIFO. 2. When the last DWORD of the data payload has been pushed, the controller core will automatically write a request into the appropriate request queue. 3. If the transaction is successful, the USB_XFERCOMPL interrupt will be generated. If the transaction is unsuccessful, an error interrupt, e.g. USB_H_NACK_n will occur.	1. Software specifies the expected size of the data packet (1 MPS) and the number of data packets (1 data packet) in the USB_DOEPSIZ_n_REG. Once the endpoint is enabled, it will wait for the host to transmit a packet to it. 2. The received packet will be pushed into the RX FIFO along with a packet status entry. 3. If the transaction is unsuccessful, e.g., due to a full RX FIFO, the endpoint will automatically NAK the incoming packet.
IN Transactions	
1. Software specifies the expected size of the data packet and the number of packets (1 data packet) in the USB_HCTSIZ_n_REG register, then enables the channel. 2. The controller core will automatically write a request into the appropriate request queue. 3. If the transaction is successful, the received data along with a status entry should be written to the RX FIFO. If the transaction is unsuccessful, an error interrupt, e.g., USB_H_NACK_n will occur.	1. Software specifies the size of the data packet and the number of data packets (1 data packet) in the USB_DIEPTSIZ_n_REG register. Once the endpoint is enabled, it will wait for the host to read the packet. 2. When the packet has been transmitted, the USB_XFERCOMPL interrupt will be generated.

When operating at the transfer level in Slave mode, one or more transaction-level operations can be pipelined thus being analogous to transfer level operation in DMA mode. Within pipelined transactions, multiple packets of the same transfer can be read/written from the FIFOs in single instance, thus preventing the need for interrupting the software on a per-packet basis.

Operating on a transfer level in Slave mode is similar to operating on the transaction-level, except the transfer size and packet count for each transfer in the USB_HCTSIZ_n_REG or USB_DOEPSIZ_n_REG register will need to be set to reflect the entire transfer. After the channel or endpoint is enabled, multiple data packets worth of payloads should be written to or read from the TX or RX FIFOs respectively (given that there is enough space or enough data).

50.5 OTG

USB OTG allows OTG devices to act in the USB Host role or the USB Device role. Thus, OTG devices will typically have a Mini-AB or Micro-AB receptacle so that it can receive an A-plug or B-plug. OTG devices will

become either an A-device or a B-device depending on whether an A-plug or a B-plug is connected.

- A-device defaults to the Host role (A-Host) whilst B-device defaults to the Device role (B-Peripheral).
- A-device and B-device may exchange roles by using the Host Negotiation Protocol (HNP), thus becoming A-peripheral and B-Host.
- A-device can turn off Vbus to save power. B-device can then wake up the A-device by requesting it to turn on Vbus and start a new session. This mechanism is called session request protocol (SRP).
- A-device always powers Vbus even if it is an A-peripheral.

OTG devices are able to determine whether they are connected to an A plug or a B plug using the ID pin of the plugs. The ID pin in A-plugs are pulled to ground whilst B-plugs have the ID pin left floating.

50.5.1 OTG Interface

The OTG_FS supports both the Session Request Protocol (SRP) and Host Negotiation Protocol (HNP) of the OTG Revision 1.3 specification. The OTG_FS controller core interfaces with the internal transceiver using the UTMI+ OTG interface. The UTMI+ OTG interface allows the controller core to manipulate the transceiver for OTG purposes, e.g., enabling/disabling pull-ups and pull-downs in HNP, and also allows the transceiver to indicate OTG related events. If an external transceiver is used instead, the UTMI+ OTG interface signals will be routed to the ESP32-P4's GPIOs instead through GPIO Matrix. Please refer to Chapter 9 [GPIO Matrix and IO MUX](#). The UTMI+ OTG interface signals are described in Table 50.5-1.

Table 50.5-1. UTMI+ OTG Interface

Signal Name	I/O	Description
usb_otg_iddig_in	I	Mini A/B Plug Indicator. Indicates whether the connected plug is mini-A or mini-B. Valid only when usb_otg_idpullup is sampled asserted. 0: Mini-A connected 1: Mini-B connected
usb_otg_avalid_in	I	A-Peripheral Session Valid. Indicates if the voltage Vbus is at a valid level for an A-peripheral session. The comparator thresholds are: 0: Vbus < 0.8 V 1: Vbus = 0.2 V to 2.0 V
usb_otg_bvalid_in	I	B-Peripheral Session Valid. Indicates if the voltage Vbus is at a valid level for a B-peripheral session. The comparator thresholds are: 0: Vbus < 0.8 V 1: Vbus = 0.8 V to 4 V
usb_otg_vbusvalid_in	I	Vbus Valid. Indicates if the voltage Vbus is valid for A/B-device/peripheral operation. The comparator thresholds are: 0: Vbus < 4.4 V 1: Vbus > 4.75 V
usb_srp_sessend_in	I	B-device Session End. Indicates if the voltage Vbus is below the B-device Session End threshold. The comparator thresholds are: 0: Vbus > 0.8 V 1: Vbus < 0.2 V

Signal Name	I/O	Description
usb_otg_idpullup	O	Analog ID input Sample Enable. Enables sampling the analog ID line. 0: ID pin sampling disabled 1: ID pin sampling enabled
usb_otg_dppulldown	O	D+ Pull-down Resistor Enable. Enables the 15 k Ω pull-down resistor on the D+ line.
usb_otg_dmpulldown	O	D- Pull-down Resistor Enable. Enables the 15 k Ω pull-down resistor on the D- line.
usb_otg_drvvbus	O	Drive Vbus. Enables driving Vbus to 5 V. 0: Do not drive Vbus 1: Drive Vbus
usb_srp_chrgvbus	O	Vbus Input Charge Enable. Directs the PHY to charge Vbus. 0: Do not charge Vbus through a resistor 1: Charge Vbus through a resistor (must be active for at least 30 ms)
usb_srp_dischrgvbus	O	Vbus Input Discharge Enable. Directs the PHY to discharge Vbus. 0: Do not discharge Vbus through a resistor. 1: Discharge Vbus through a resistor (must be active for at least 50 ms).

50.5.2 ID Pin Detection

Bit USB_CONIDSTS in register USB_GOTGCTL_REG indicates whether the OTG controller is an A-device (1'b0) or a B-device (1'b1). The USB_CONIDSTSCHNG interrupt will trigger whenever there is a change to USB_CONIDSTS, i.e., when a plug is connected or disconnected.

50.5.3 Session Request Protocol (SRP)

50.5.3.1 A-Device SRP

Figure 50.5-1 illustrates the flow of SRP when the OTG_FS is acting as an A-device, i.e., default host and the device that powers Vbus.

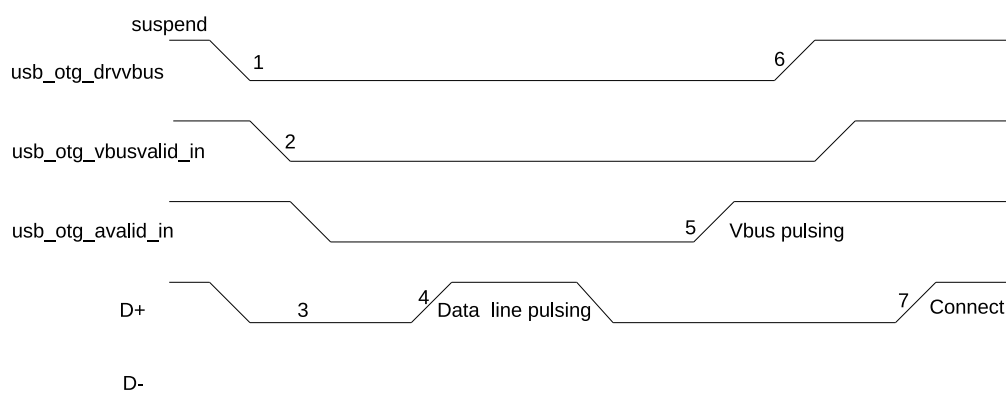


Figure 50.5-1. A-Device SRP

- To save power, the application suspends and turns off port power when the bus is idle by writing to the Port Suspend (USB_PRTSUSP to 1'b0) and Port Power (USB_P RTPWR to 1'b0) bits in the Host Port Control and Status register.

2. PHY indicates port power off by deasserting the `usb_otg_vbusvalid_in` signal.
3. The A-device must detect SEO for at least 2 ms to start SRP when Vbus power is off.
4. To initiate SRP, the B-device turns on its data line pull-up resistor for 5 to 10 ms. The OTG_FS core detects data-line pulsing.
5. The device drives Vbus above the A-device session valid (2.0 V minimum) for Vbus pulsing. The OTG_FS core interrupts the application on detecting SRP. The Session Request Detected bit (`USB_SESSREQINT`) is set in Global Interrupt Status register.
6. The application must service the Session Request Detected interrupt and turn on the Port Power bit by writing the Port Power bit in the Host Port Control and Status register. The PHY indicates port power-on by asserting `usb_otg_vbusvalid_in` signal.
7. When the USB is powered, the B-Device connects, completing the SRP process.

50.5.3.2 B-Device SRP

Figure 50.5-2 illustrates the flow of SRP when the OTG_FS is acting as a B-device, i.e., does not power Vbus.

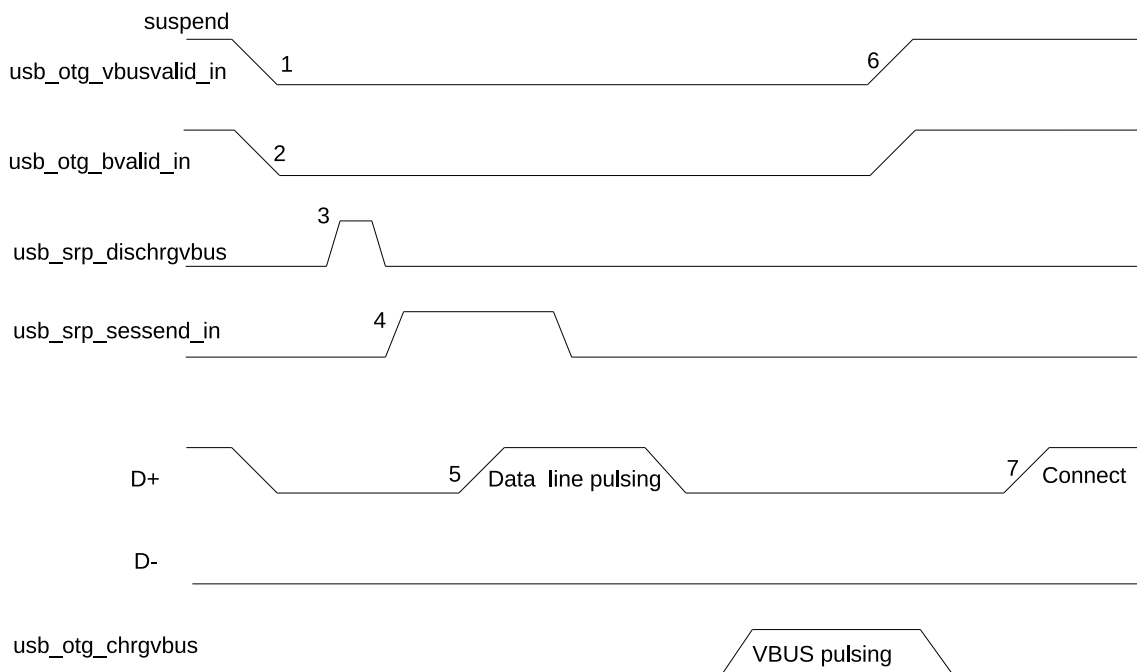


Figure 50.5-2. B-Device SRP

1. To save power, the host (A-device) suspends and turns off port power when the bus is idle. PHY indicates port power off by deasserting the `usb_otg_vbusvalid_in` signal. The OTG_FS core sets the Early Suspend bit (`USB_ERLYSUSP`) in the Core Interrupt register after detecting 3 ms of bus idleness. Following this, the OTG_FS core sets the USB Suspend bit (`USB_USBSUSP`) in the Core Interrupt register. PHY indicates the end of the B-device session by deasserting the `usb_otg_bvalid_in` signal.
2. The OTG_FS core asserts the `usb_otg_dischrgvbus` signal to indicate to the PHY to speed up Vbus discharge.
3. PHY indicates the session's end by asserting the `usb_otg_sessend_in` signal. This is the initial condition

for SRP. The OTG_FS core requires 2 ms of SEO before initiating SRP. For a USB 2.0 full-speed serial transceiver, the application must wait until Vbus discharges to 0.2 V after USB_BSESVDL is deasserted.

4. The application waits for 1.5 seconds (TB_SEO_SRP time) before initiating SRP by writing the Session Request bit (USB_SESREQ) in the OTG Control and Status register. The OTG_FS core performs data-line pulsing followed by Vbus pulsing.
5. The host (A-device) detects SRP from either the data-line or Vbus pulsing, and turns on Vbus. The PHY indicates Vbus power-on by asserting `usb_otg_vbusvalid_in`.
6. The OTG_FS core performs Vbus pulsing by asserting `usb_srp_chrgvbus`. The host (A-device) starts a new session by turning on Vbus, indicating SRP success. The OTG_FS core interrupts the application by setting the Session Request Success Status Change bit (USB_SESREQSC) in the OTG Interrupt Status register. The application reads the Session Request Success bit in the OTG Control and Status register.
7. When the USB is powered, the OTG_FS core connects, completing the SRP process.

50.5.4 Host Negotiation Protocol (HNP)

50.5.4.1 A-Device HNP

Figure 50.5-3 illustrates the flow of HNP when the OTG_FS is acting as an A-device.

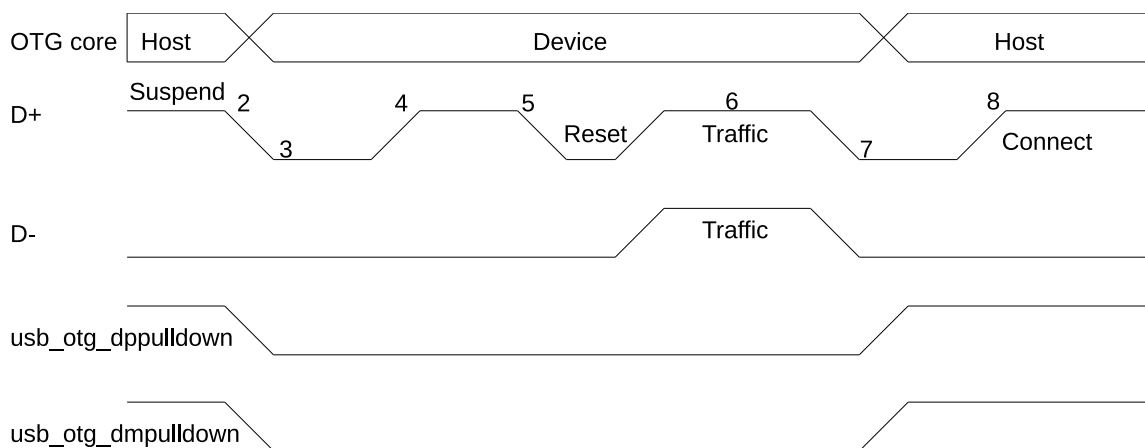


Figure 50.5-3. A-Device HNP

1. The OTG_FS core sends the B-device a SetFeature `b_hnp_enable` descriptor to enable HNP support. The B-device's ACK response indicates that the B-device supports HNP. The application must set Host Set HNP Enable bit (USB_HSTSETHNPEN) in the OTG Control and Status register to indicate to the OTG_FS core that the B-device supports HNP.
2. When it has finished using the bus, the application suspends by writing the Port Suspend bit (USB_PRTSUSP) in the Host Port Control and Status register.
3. When the B-device observes a USB suspend, it disconnects, indicating the initial condition for HNP. The B-device initiates HNP only when it must switch to the host role; otherwise, the bus continues to be suspended. The OTG_FS core sets the Host Negotiation Detected interrupt (USB_HSTNEGDET) in the OTG Interrupt Status register, indicating the start of HNP. The OTG_FS core deasserts the `usb_otg_dppulldown` and `usb_otg_dmpulldown` signals to indicate a device role. The PHY enables the D+ pull-up resistor, thus indicates a connection for the B-device. The application must read the Current

Mode bit (USB_CURMOD_INT) in the OTG Control and Status register to determine Device mode operation.

4. The B-device detects the connection, issues a USB reset, and enumerates the OTG_FS core for data traffic.
5. The B-device continues the host role, initiating traffic, and suspends the bus when done. The OTG_FS core sets the Early Suspend bit (USB_ERLYSUSP) in the Core Interrupt register after detecting 3 ms of bus idleness. Following this, the OTG_FS core sets the USB Suspend bit (USB_USBSUSP) in the Core Interrupt register.
6. In Negotiated mode, the OTG_FS core detects the suspend, disconnects, and switches back to the host role. The OTG_FS core asserts the `usb_otg_dppulldown` and `usb_otg_dmpulldown` signals to indicate its assumption of the host role.
7. The OTG_FS core sets the Connector ID Status Change bit (USB_CONIDSTS) in the OTG Interrupt Status register. The application must read the connector ID status in the OTG Control and Status register to determine the OTG_FS core's operation as an A-device. This indicates the completion of HNP to the application. The application must read the Current Mode bit (USB_CURMOD_INT) in the OTG Control and Status register to determine Host mode operation.
8. The B-device connects, completing the HNP process.

50.5.4.2 B-Device HNP

Figure 50.5-4 illustrates the flow of HNP when the OTG_FS is acting as a B-device.

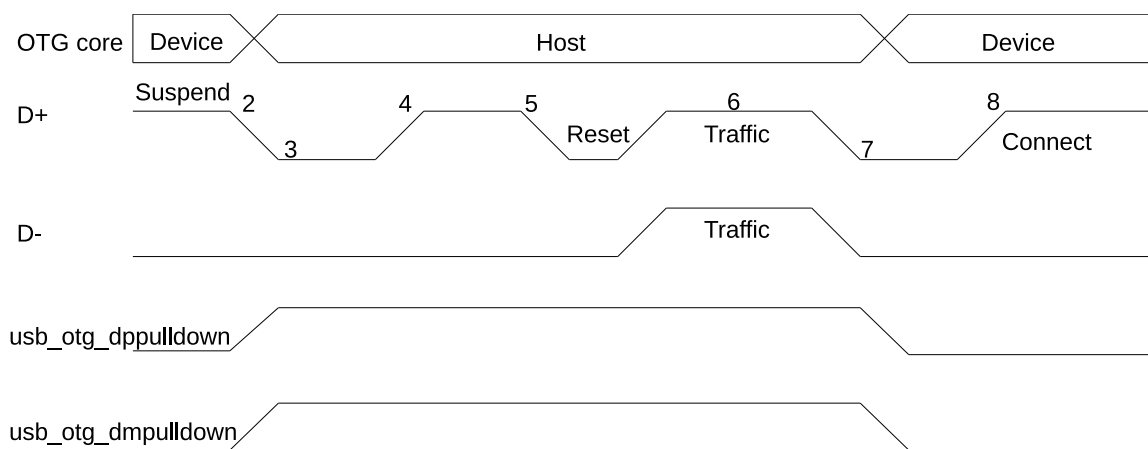


Figure 50.5-4. B-Device HNP

1. The A-device sends the SetFeature `b_hnp_enable` descriptor to enable HNP support. The OTG_FS core's ACK response indicates that it supports HNP. The application must set the Device HNP Enable bit (USB_DEVHNPEN) in the OTG Control and Status register to indicate HNP support. The application sets the HNP Request bit (USB_DEVHNPEN) in the OTG Control and Status register to indicate to the OTG_FS core to initiate HNP.
2. When A-device has finished using the bus, it suspends the bus.
 - (a) The OTG_FS core sets the Early Suspend bit (USB_ERLYSUSP) in the Core Interrupt register after 3 ms of bus idleness. Following this, the OTG_FS core sets the USB Suspend bit (USB_USBSUSP) in

- the Core Interrupt register. The OTG_FS core disconnects and the A-device detects SEO on the bus, indicating HNP.
- (b) The OTG_FS core asserts the `usb_otg_dppulldown` and `usb_otg_dmpulldown` signals to indicate its assumption of the host role.
 - (c) The A-device responds by activating its D+ pull-up resistor within 3 ms of detecting SEO. The OTG_FS core detects this as a connect.
 - (d) The OTG_FS core sets the Host Negotiation Success Status Change interrupt in the OTG Interrupt Status register (`USB_CONIDSTS`), indicating the HNP status. The application must read the Host Negotiation Success bit (`USB_HSTNEGSCS`) in the OTG Control and Status register to determine host negotiation success. The application must read the Current Mode bit (`USB_CURMOD_INT`) in the Core Interrupt register to determine Host mode operation.
3. Program the `USB_PRTTWR` bit to 1'b1. This drives Vbus on the USB.
 4. Wait for the `USB_PRTCONDET` interrupt. This indicates that a device is connected to the port.
 5. The application sets the reset bit (`USB_PRTTWR`) and the OTG_FS core issues a USB reset and enumerates the A-device for data traffic.
 6. Wait for the `USB_PRTENCHNG` interrupt.
 7. The OTG_FS core continues the host role of initiating traffic, and when done, suspends the bus by writing the Port Suspend bit (`USB_PRTSUSP`) in the Host Port Control and Status register.
 8. In Negotiated mode, when the A-device detects a suspend, it disconnects and switches back to the host role. The OTG_FS core deasserts the `usb_otg_dppulldown` and `usb_otg_dmpulldown` signals to indicate the assumption of the device role.
 9. The application must read the Current Mode bit (`USB_CURMOD_INT`) in the Core Interrupt register to determine the Host mode operation.
 10. The OTG_FS core connects, completing the HNP process.

50.6 Registers

The catalog and comprehensive specifications of USB OTG registers are subject to a Non-Disclosure Agreement (NDA) as mandated by the IP provider. To obtain support information for a particular register, please contact Espressif Technical Support Team via [Technical Inquires](#).